Program and Branch Name: BTech Computer Science and Engineering (Data Science)

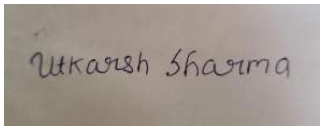
PBL Progress Report – Final

Course Name	**Object Oriented Techniques using Java**
Course Code	CCSE0355
Faculty Name	Mr. Archit
Assignment Type	Capstone Project
Student(s) Name	1. Utkarsh Sharma 2. Yugansh Goyal 3. Vishesh 4. Vishal Tiwari 5. Yash Raj Sahu
ERP ID (I Year) / Roll No. (II Year onwards)	1. 2501331540283 2. 2501331540299 3. 2501331540294 4. 2501331540293 5. 2501331540297
Team/Group No.	Group 59
Problem Statement / Project Title	Alumini Engagement and Networking System
SDG(s)	Quality Education
Reporting Month	Month 3 – Final
Date of Submission	24/10/2026

Student Declaration

We certify that this capstone project is our own work, with all sources appropriately acknowledged. We understand that a 0% mark and further academic penalties may apply if the content is found to be plagiarised or inappropriately AI-generated.

Student Name & Signature(s): Utkarsh Sharma



Date: 24/10/2026

Project Title: Alumini Engagement and Networking System

2. Student Details / Team Details

Student Name	ERP ID / Roll No.	Role / Contribution
Utkarsh Sharma	2501331540283	Implementation and local checks; confirm wording with team.
Yugansh Goyal	2501331540299	Individual role not recorded in the available project notes.
Vishesh	2501331540294	Individual role not recorded in the available project notes.
Vishal Tiwari	2501331540293	Individual role not recorded in the available project notes.
Yash Raj Sahu	2501331540297	Individual role not recorded in the available project notes.

Programme: BTech Computer Science and Engineering (Data Science) | Course: Object Oriented Techniques using Java | Course Code: CCSE0355 | Faculty: Mr. Archit | Group 59 | SDG: Quality Education.

3. Executive Summary

The Alumini Engagement and Networking System is a Java project for keeping alumni and student profiles in one searchable directory. It started as a console program for adding, listing, and searching profiles. The later work added file storage, a TCP service for listing and searching profiles, and a Swing desktop interface. We checked that saved profiles were still available after restarting the app, that older records could still be read, and that invalid input was rejected. The project is a local prototype for learning and demonstration; it is not yet connected to NIET systems or ready for public use.

4. Introduction

Alumni details can be spread across spreadsheets, messages, and different online platforms. This can make it harder for students to find graduates working in a field they are interested in. The project explores a simple way to keep alumni and student details together. It also gave us a chance to apply Java OOTS topics such as inheritance, encapsulation, file handling, exception handling, sockets, and Swing. The work is limited to a local prototype and does not include a live institutional database.

5. Problem Definition

The project aims to provide one place to register and search alumni and student profiles. A profile stores common details such as name, graduation year, area or expertise, and email, along with fields specific to the person’s role. The application can save profiles locally and

supports read-only listing and search over TCP. It does not currently provide login, encrypted network communication, profile editing or deletion, or integration with an institutional portal.

The intended users are alumni, current students, and staff who may coordinate alumni or placement activities. The current build has not been deployed for those users and has not been connected to a college portal. The project therefore demonstrates the core directory functions rather than claiming that a complete alumni engagement service is already available.

6. Industry / Application Overview

Alumni information is often managed through spreadsheets, college portals, social networks, or dedicated alumni platforms. This project does not compare those products or claim to replace them. Instead, it builds a small example of how a profile directory can work and how it can be extended with storage, search, and a desktop interface. Before a system like this could be used with real alumni, it would need proper access controls, privacy rules, security testing, and feedback from users.

7. Module 1: Object-Oriented Profile Modelling and Console Workflows

In the first version, User held the fields shared by alumni and students, while Alumni and Student added their own details. ProfileDirectory kept the records in memory and provided registration, alphabetical listing, and name search. This gave the project a basic working structure using classes, constructors, encapsulation, inheritance, and polymorphism. At this stage, the records were lost when the program closed. [1], [2].

The first implementation also included a menu for choosing alumni or student registration and for viewing or searching the directory. Required text fields were checked for blank input, and non-numeric menu input was handled so the program could continue instead of stopping unexpectedly. The directory used an in-memory collection, so this version was suitable for demonstrating the main workflows but did not retain records after exit.

The early workflow was checked by compiling and running the console application and trying registration, alphabetical listing, and partial-name search. These were manual checks; they were not presented as automated tests. At this point, the main result was a working base that later modules could extend.

The Report I screenshot shows the original console menu, profile registration, and alphabetical listing.

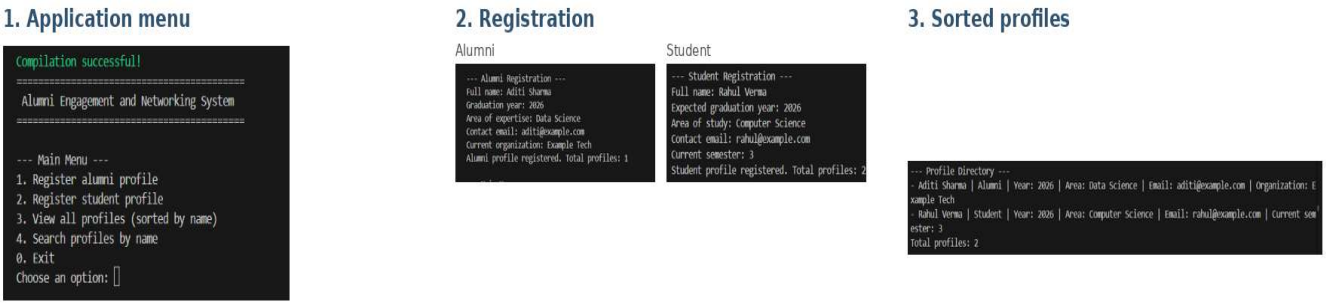


Figure 1: Original console evidence montage from Report I (menu, registration, and sorted profiles).

8. Module 2: Input Handling and Directory Operations

The console version allowed users to register alumni or students, list profiles alphabetically, and search by a full or partial name without case sensitivity. Shared fields were kept in the parent class, while the subclasses held role-specific fields. These early workflows were checked manually; the original Report I did not include automated tests for them.

9. Module 3: Persistent Storage and Exception Handling

ProfileStorage saves and loads records from data/profiles.csv using UTF-8. Fields are encoded so commas and non-English characters do not break the record format. If the file does not exist, the application starts with an empty directory. If a record is malformed, the code reports an IOException instead of silently accepting it. Saving uses a temporary file before moving it into place. The loader also recognizes older records with IDs and keeps those IDs; details that could not be recovered reliably are shown as “Not provided.” [4], [5], [9].

The saved records use a type marker to distinguish alumni from student entries, followed by encoded fields. Encoding each field avoids problems when a value contains commas or non-ASCII characters. The storage tests checked both alumni and student round-trips, including punctuation and Unicode, loading when the file is missing, handling a malformed record, and keeping legacy IDs when those records are loaded and saved again.

The write path creates a temporary file before moving it into place, reducing the chance of leaving a partly written data file if saving is interrupted. The implementation attempts an atomic move where supported. This is local file storage, not a database, and the application has not been tested for concurrent edits by multiple users.

10. Module 4: TCP Directory Service

DirectoryServer accepts TCP connections and supports two read-only commands: LIST and SEARCH <name>. DirectoryClient sends these commands and reads the server response. The service is for local demonstration only; it does not use login checks or encrypted connections. Smoke tests checked that LIST and SEARCH returned the expected responses. [6], [8].

The client sends a supported command and reads the response until the protocol terminator. LIST returns the directory entries, while SEARCH returns entries matching the requested name; a search with no match returns a no-match response. The socket smoke test exercised both commands against the running server. The network feature demonstrates client/server communication, but it is not intended for use over an untrusted network.

Console directory after reload

```
--- Profile Directory ---
- Test Alumni | Alumni | Year: ALU2026001 | Area: alumni.test@example.com | Email: alumni
.test@example.com | Organization: Example Technologies
- Test Student | Student | Year: STU2026001 | Area: student.test@example.com | Email: Com
puter Science | Current semester: 2
Total profiles: 2

--- Main Menu ---
1. Register alumni profile
2. Register student profile
3. View all profiles (sorted by name)
4. Search profiles by name
0. Exit
Choose an option:
```

TCP client LIST and SEARCH results

```
lumini-Engagement-and-Networking-System-Java-
>> java -cp out com.alumni.DirectoryClient
Connected to alumni directory at localhost:5050
Commands: LIST, SEARCH <name>, EXIT
> LIST
Test Alumni | Alumni | Year: ALU2026001 | Area: alumni.test@example.com | Email: alumni.t
est@example.com | Organization: Example Technologies
Test Student | Student | Year: STU2026001 | Area: student.test@example.com | Email: Compu
ter Science | Current semester: 2
> SEARCH Test Alumni
Test Alumni | Alumni | Year: ALU2026001 | Area: alumni.test@example.com | Email: alumni.t
est@example.com | Organization: Example Technologies
> SEARCH Nobody Example
No matching profiles.
> EXIT
PS C:\Users\Roger\Desktop\Alumini-Engagement-and-Networking-System-Java->
```

Figure 2: Original Report II terminal evidence for persistence and TCP LIST/SEARCH checks.

11. Module 5: Desktop Interface, Validation and Legacy Compatibility

The Swing application shows saved profiles in a desktop window. The list can be filtered as the user types, and the Clear and Show all actions reset the view. A registration dialog is available for alumni and students. The form checks that graduation years are between 1900 and 2100, email addresses follow a basic format, and student semester values are between 1 and 8. The loader also keeps the older IDs ALU2026001 and STU2026001. Where the older records did not contain reliable values, the app shows “Not provided” rather than guessing. [7], [12].

This module brings the earlier console and storage work into a window-based interface. It uses the same saved profile data rather than maintaining a separate list only for the screen. The manual restart check was used to verify that the interface loaded the records from storage again, rather than showing only the profile created during the current session.

For a manual check, a profile named Aman was added with graduation year 2026, area Data Science, email aman@mail.com, and organization Data Analyst. We closed and reopened the application, and all three profiles were still visible.

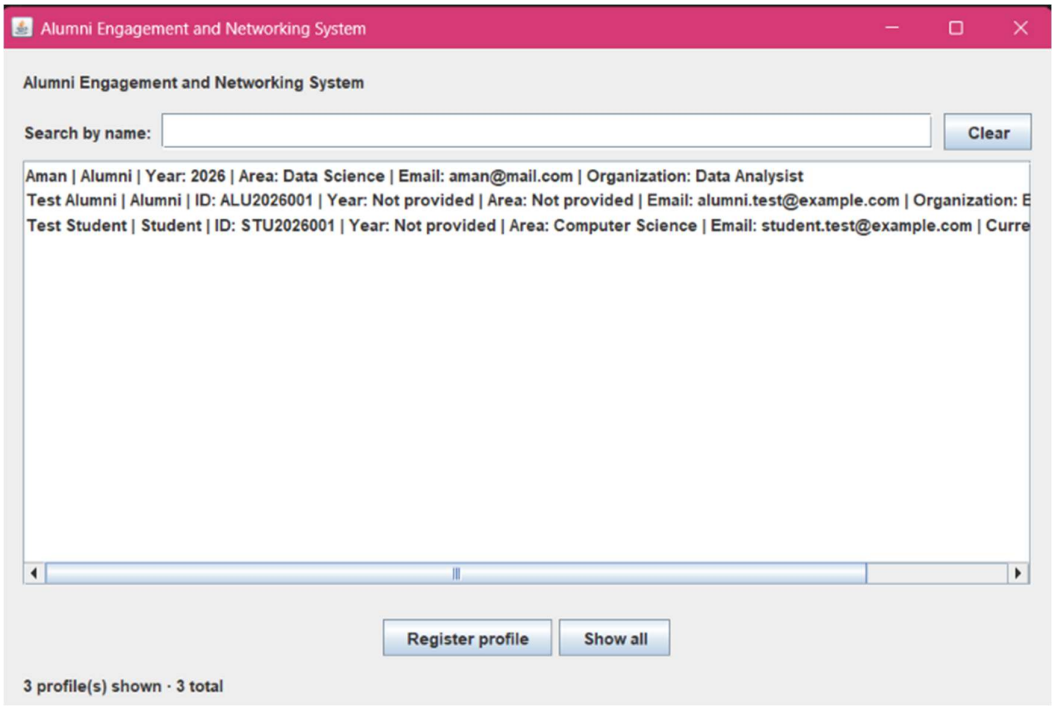


Figure 3: Swing desktop application after restart, showing Aman and the two legacy profiles.

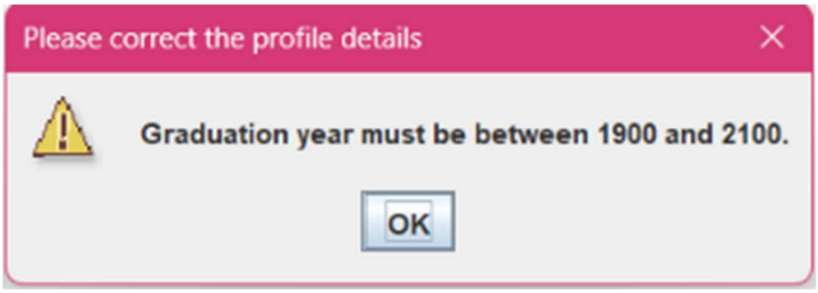


Figure 4: GUI validation message rejecting graduation year 1899.

12. Final Integrated Solution / Testing and Validation

The final prototype brings together the profile classes (User, Alumni, and Student), ProfileDirectory, ProfileStorage, DirectoryServer, DirectoryClient, and DesktopApp. The following local checks passed:

The checks were chosen to cover the main risks introduced by the later modules: whether stored data can be read back correctly, whether unusual characters survive a save/load cycle, whether malformed input is reported, whether legacy records remain identifiable, and whether the network commands return useful responses. The GUI was also checked manually after closing and reopening the application.

- ProfileStorageSmokeTest: missing-file behavior, alumni/student round-trip with punctuation and Unicode, legacy ID load and round-trip, and malformed-record handling.
- ProfileValidationSmokeTest: valid profiles accepted; invalid graduation years, email addresses, and student semesters rejected.

- DirectoryServerSmokeTest: TCP LIST and SEARCH smoke tests passed.
- Manual GUI persistence check: three profiles remained present after restarting the application.
- Manual negative validation check: year 1899 was rejected with “Graduation year must be between 1900 and 2100.”

These checks cover the main project workflows, but they are not a full security, load, or usability evaluation. The TCP service is read-only and has no authentication or encryption.

13. Expected Benefits / Learning Outcomes

The directory gives users a simple way to view and search the profiles entered for a demonstration.

Saved profiles remain available after the application is closed and reopened.

The class structure shows how abstraction, inheritance, encapsulation, and polymorphism can be used in a Java project.

The desktop interface uses Swing components, event listeners, and form validation.

The smoke tests cover file input/output, exception handling, and communication between a TCP client and server.

The project relates to SDG 4 (Quality Education) because a directory like this could help students discover alumni and career paths. This benefit has not yet been tested with real users.

14. Challenges Faced

One issue was that older local records used ID-like values in a field now treated as a graduation year. The loader was adjusted to preserve those IDs and show missing details as “Not provided.” Validation was added for graduation year, email, and semester values. The application still needs profile editing and deletion, and the network service needs authentication and encryption before it could be used outside a local demonstration. It has not been deployed or tested with real users.

15. Future Improvements

- Add safe profile update and delete workflows with confirmation and storage regression tests.
- Improve field-level validation messages, duplicate-profile handling, and keyboard accessibility.
- Expand tests for search edge cases, storage write failures, malformed versioned records, and concurrent requests.
- Add authentication and transport security before network use beyond local demonstration.

- Consider database-backed storage only after privacy, access-control, and operational requirements are defined.
- Conduct usability testing with consent and appropriate test data before making claims about stakeholder impact.

16. Conclusion

The project grew from a console program into a local Java application with file storage, a read-only TCP service, and a Swing interface. The storage, validation, and TCP smoke tests passed. Manual checks also showed that a new profile remained after restart, older IDs were preserved, and an invalid graduation year was rejected. The next steps would be to add safe editing and deletion, improve security, and test the application with users before considering wider use.

17. References

[1] Oracle, “Object-Oriented Programming Concepts,” The Java Tutorials. https://docs.oracle.com/javase/tutorial/java/concepts/ (accessed 24 October 2026).

[2] Oracle, “Inheritance,” The Java Tutorials. https://docs.oracle.com/javase/tutorial/java/IandI/subclasses.html (accessed 24 October 2026).

[3] Oracle, “Handling Errors with Exceptions,” The Java Tutorials. https://docs.oracle.com/javase/tutorial/essential/exceptions/ (accessed 24 October 2026).

[4] Oracle, “Basic I/O,” The Java Tutorials. https://docs.oracle.com/javase/tutorial/essential/io/ (accessed 24 October 2026).

[5] Oracle, “Reading, Writing, and Creating Files,” Java SE 17 API Specification. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/nio/file/Files.html (accessed 24 October 2026).

[6] Oracle, “Sockets,” The Java Tutorials. https://docs.oracle.com/javase/tutorial/networking/sockets/ (accessed 24 October 2026).

[7] Oracle, “Creating a GUI With Swing,” The Java Tutorials. https://docs.oracle.com/javase/tutorial/uiswing/ (accessed 24 October 2026).

[8] Oracle, “Java SE 17 API Specification: java.net,” https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/net/package-summary.html (accessed 24 October 2026).

[9] Oracle, “Java SE 17 API Specification: java.util.Base64,” https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Base64.html (accessed 24 October 2026).

[10] GitHub, “GitHub Actions Documentation.” https://docs.github.com/en/actions (accessed 24 October 2026).

[11] GitHub, “Alumini Engagement and Networking System Java repository.” https://github.com/shutkarshX/Alumini-Engagement-and-Networking-System-Java- (accessed 24 October 2026).

[12] Oracle, “Java SE 17 API Specification: javax.swing,” https://docs.oracle.com/en/java/javase/17/docs/api/java.desktop/javax/swing/package-summary.html (accessed 24 October 2026).

18. Appendix – Source Code

Source code repository: https://github.com/shutkarshX/Alumini-Engagement-and-Networking-System-Java-

The project repository contains the Java source files, smoke-test classes, and GitHub Actions workflow used for verification. Local profile data and its backup are not included in the public repository. The repository link is provided here so the source can be reviewed; the appendix page limit should be followed if source-code pages are added. [10], [11].